COL 11D1
LECTURE - 3

JULY 28, 2026

LAST CLASS

1. Selection sort

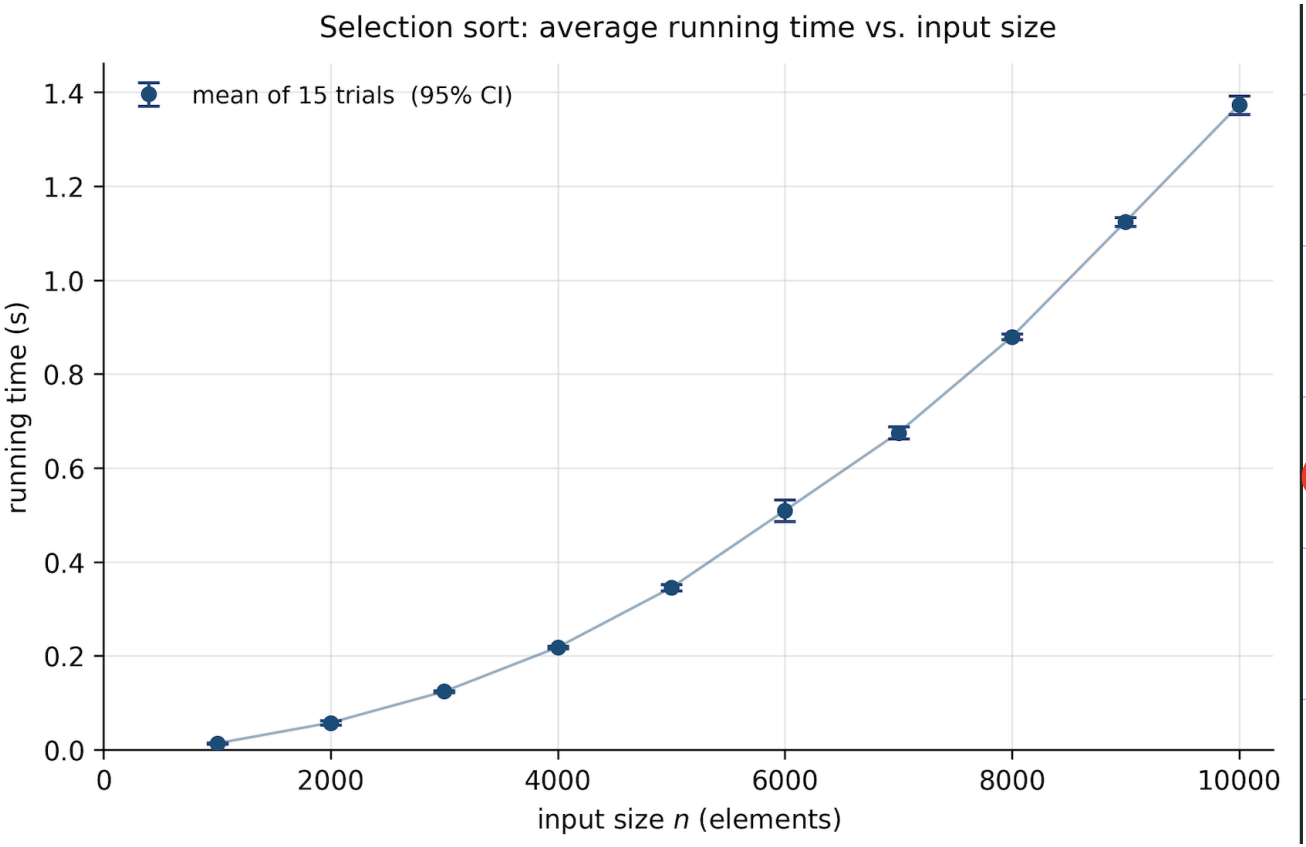
2. How to prove correctness?
   Loop invariants

3. How to analyze runtime complexity?

LAST CLASS: How to analyze runtime complexity?

Approach #1: Experimental Analysis

Idea: Log the run time on different inputs



① Machine specific: Comparison of algos becomes challenging

② Knows only about inputs we test on

③ Time consuming

Need to implement the algorithm

LAST CLASS: How to analyze runtime complexity?

Desired goal: A general methodology that quantifies how the run time scales as the size of input

LAST CLASS: How to analyze runtime complexity?

Desired goal: A general methodology that quantifies how the run time scales as the size of input

Design constraints

- uses a high-level description of the algorithm
- takes into account all possible inputs
- is independent of hardware and software environment

LAST CLASS: How to analyze runtime complexity?

Desired goal: A general methodology that quantifies how the run time scales as the size of input

- uses a high-level description of the algorithm
- takes into account all possible inputs
- is independent of hardware and software environment



Idea: Count the number of primitive operations
→ indivisible operations (for practical purposes) - that take constant amount of time

e.g. - arithmetic operations (add, subtract, multiply)
- logical operations (comparisons)
- function call, return
- assignment, etc.

32-bit
64-bit

SIDE NOTE

Are arithmetic operations constant time, independent of input?

NO in practical these operations depend on input size, esp. if input is very large

Assumption: Read about WORD RAM model.

Example: Finding smallest element in an array.

Count the number of primitive operations or high-level description of the algorithm



```
def find_minimum(a):
    min = a[0]
    n = len(a)
    for i in range(0, n):
        if a[i] < min:
            min = a[i]
    return min
```

pseudo code

→ more structure than prose
→ less formal than code

```
FIND_MIN(a)
    n ← size(a)
    min ← a[0]
    for i ∈ {1, 2, .. n}
        if a[i] < min
            min = a[i]
    return min
```

How Runtime Scales As Input Size Grows

Count the number of primitive operations or high-level description of the algorithm



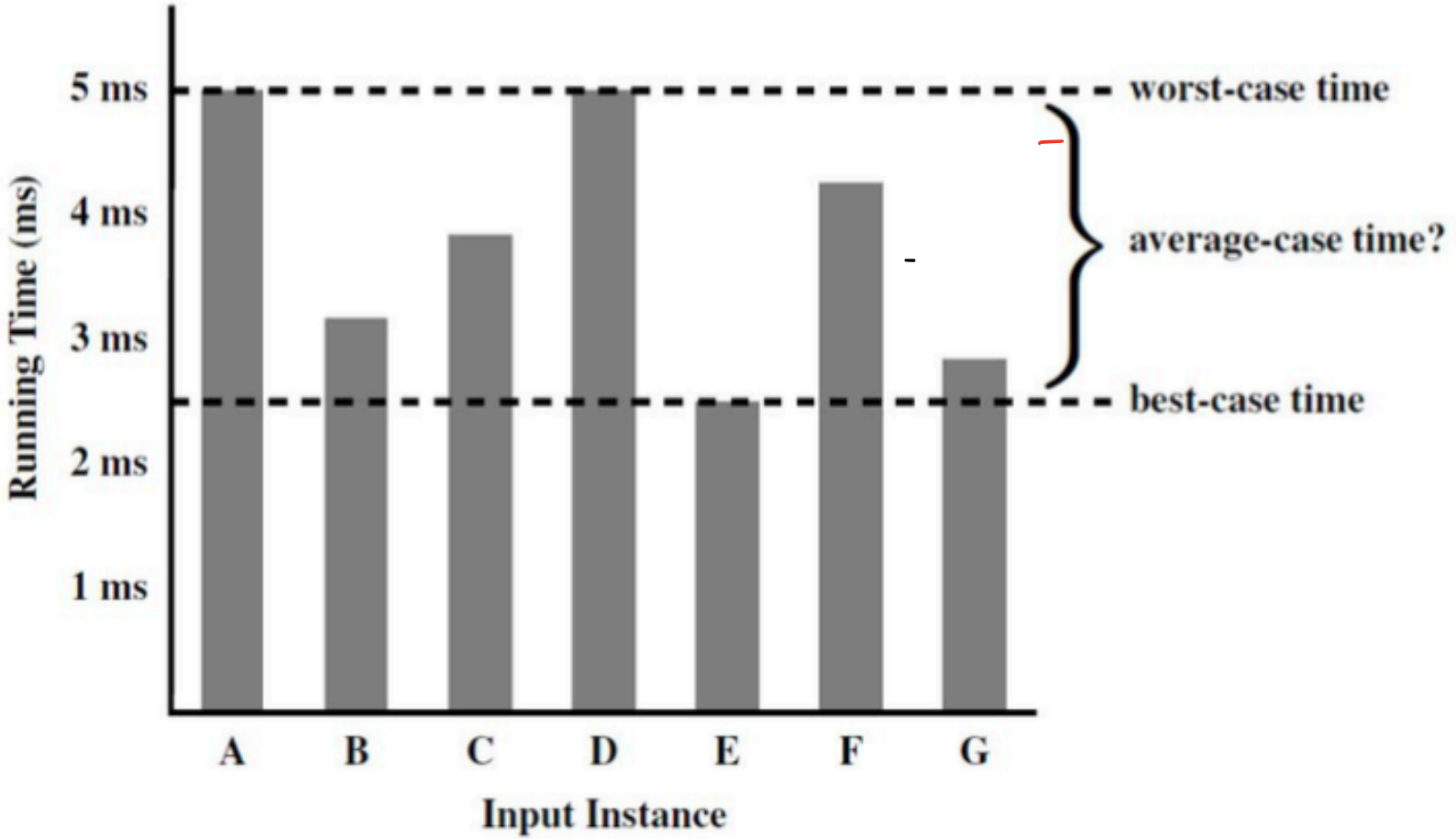
for what input?

How Runtime Scales As Input size Grows

Count the number of primitive operations or high-level description of the algorithm

for what input?





How Runtime Scales As Input Size Grows



Count the number of primitive operations or high-level description of the algorithm for worst-case input

Why worst case?

- gives upper bound on performance

- easier to calculate compared to average time

LET'S COUNT # OF PRIMITIVE OPERATIONS

Example: Find the max element in an array

```
def find_minimum(a):
    min = a[0]                  1
    n = len(a)                  1
    for i in range(0, n):       n
        if a[i] < min:          n
            min = a[i]          n
    return min                  1
```

c_1 → initialization
c_2 → memory access → take
c_3 → comparison
c_4 → return

Assume constant amount of time + one unit of time

TOO COMPLICATED

$\approx 3n+3$ primitive steps

Q: Can we simplify this further

Total
$i = 1$
while $i < n$

$i += 1$

CAN WE SIMPLIFY FURTHER?

of steps $\sim 3n+3$ → machine dependent

GOAL: How does runtime scale as input size grows?

Asymptotic analysis

→ Focus on large value of n, I can ignore the effect of constant factors & lower-order terms

	contribution	
$n = 1$	50%	$3n+3$ grows
$n = 100$	~1%	like n
$n = 100,00$	~0.01%	$3n+3$ grows like $3n$

ASYMPTOTIC ANALYSIS:

CARE ABOUT LARGE VALUES OF n

Why?

↳ This is where real difference comes in terms of algorithm efficiency

↳ large value of n are actually important in real-world

→ **Sweet Spot:**

① Sharp enough to compare algos

② Coarse enough to avoid environment specific details

Algo 1 : $100 \log_2 n$

Algo : n^2

For large values of n,

$100 \log_2 n$ is faster than n^2

ASYMPTOTIC ANALYSIS:

CARE ABOUT LARGE VALUES OF n

environment-specific

Suppress constant factors and lower-order terms

irrelevant for large values of n

$\log_2 n + 3$ grows like $\log_2 n$

$n^2 + n$ grows like n^2

ASYMPTOTIC ANALYSIS

Principles

1. Count # of primitive operations
2. Consider worst case input
3. Ignore constant factors and lower order terms

- Algorithm 1 → $100n + 10000$ ✓
- Algo 2 : $n \log_2 n$

EXAMPLE: SELECTION SORT

```
def selection_sort(a):
    n = len(a)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        a[i], a[min_idx] = a[min_idx], a[i]
```

$\sim c_1 n^2 + c_2 n + c_3$

grows like n^2

is $O(n^2)$

"big Oh" of n^2

"order" of n^2

$n-1$

$n-1$

1

1

$n-1$

$\sum_{i=0}^{n-2} n-(i+1)$

$\sim n^2$

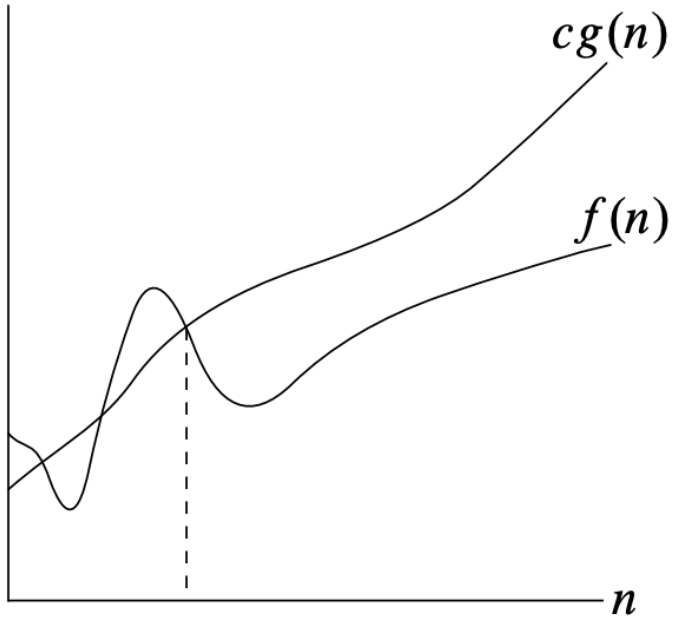
1

BIG OH DEFINITION

Definition: Let $f, g : \mathbb{N} \to \mathbb{R}^+$ be functions

f is $O(g)$ if $\exists\, c > 0$: $f(n) \leq c\,g(n)$ $\forall\, n \geq n_0$

$n_0 \in \mathbb{N}$ is a constant



$f(n) = O(g(n))$

EXAMPLES

n is $O(n^2)$

Claim

$2n+6$ is $O(n)$

Proof: , c and n_0 such that $\boxed{2n+6 \leq c(n)}$

$\forall \; n \geq n_0$

$c = 3$, $n_0 = 6$

Examples

$100n$ is $O(n^2)$

$c = 100$

$n_0 = 1$

2^{n+2} is $O(2^n)$

$c = 5$

$n_0 \geq 1$

Examples

polynomial

Claim: if $f(n) = a_k n^k + a_{k-1} n^{k-1} + \ldots + a_0$, then

$f(n)$ is $O(n^k)$

$c = \sum_{i=0}^{k} a_i$, $n_0 = 1$

a_i can be -ve

$f(n) \leq c n^k$

$c = \sum_{i=0}^{k} |a_i|$ $\quad n_0 = 1$

Take home

Examples

Claim: If $f(n) = n^k$ then $f(n)$ is NOT $O(n^{k-1})$

Examples → set

Claim: If $f(n)$ is $O(g(n))$ and $g(n)$ is $O(h(n))$ then $f(n)$ is $O(h(n))$

Proof: $f(n) \leq c_1 g(n)$ ← $\exists c_1, n_1$ such that

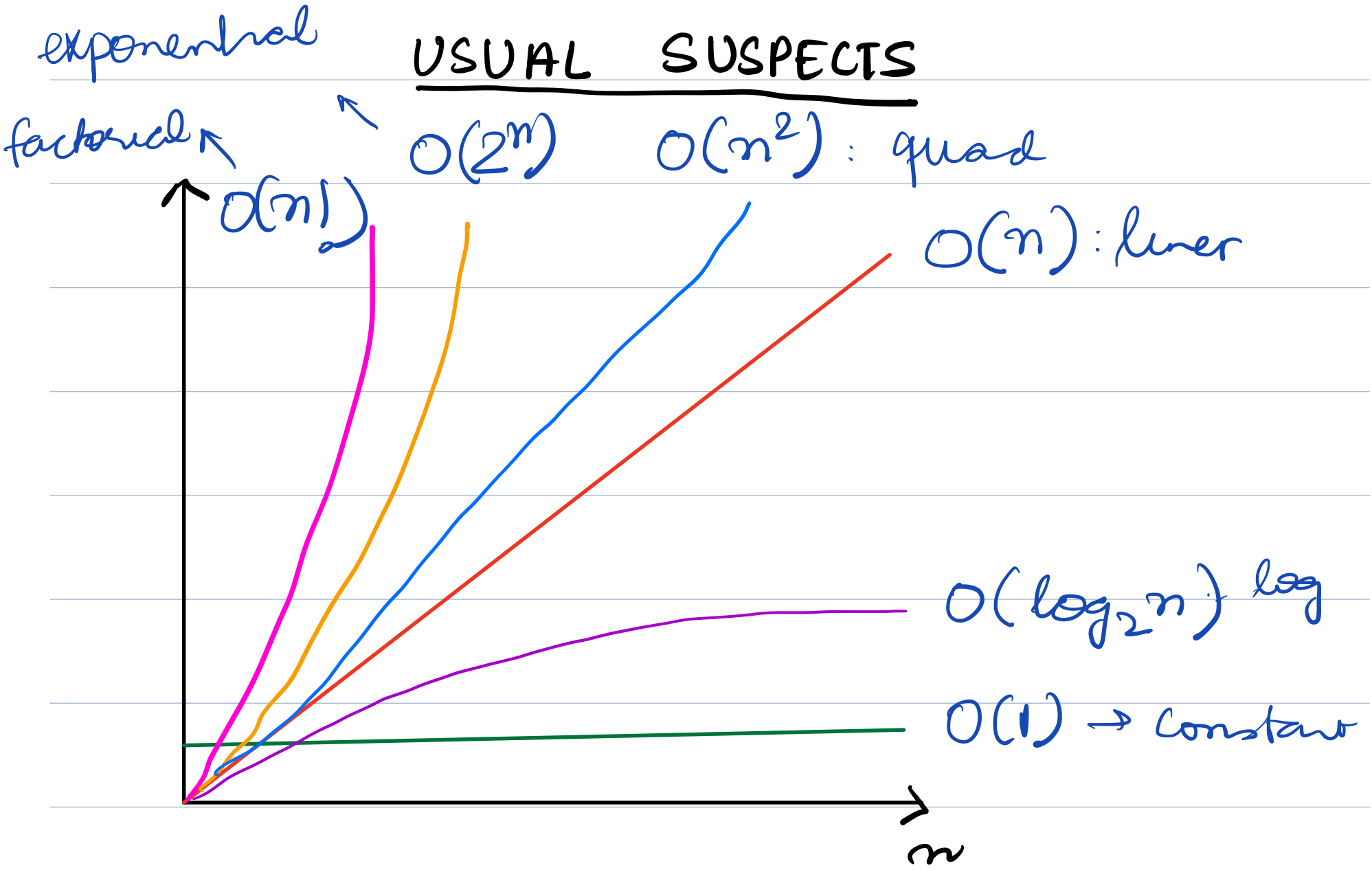
$\exists c_2, n_2$: $g(n) \leq c_2 h(n)$

$c = c_1 c_2$ $\quad$ $n_0 = \max(n_1, n_2)$

$f(n) \leq c_1 c_2 h(n)$

$f(n)$ is $O(h(n))$

USUAL SUSPECTS



Set

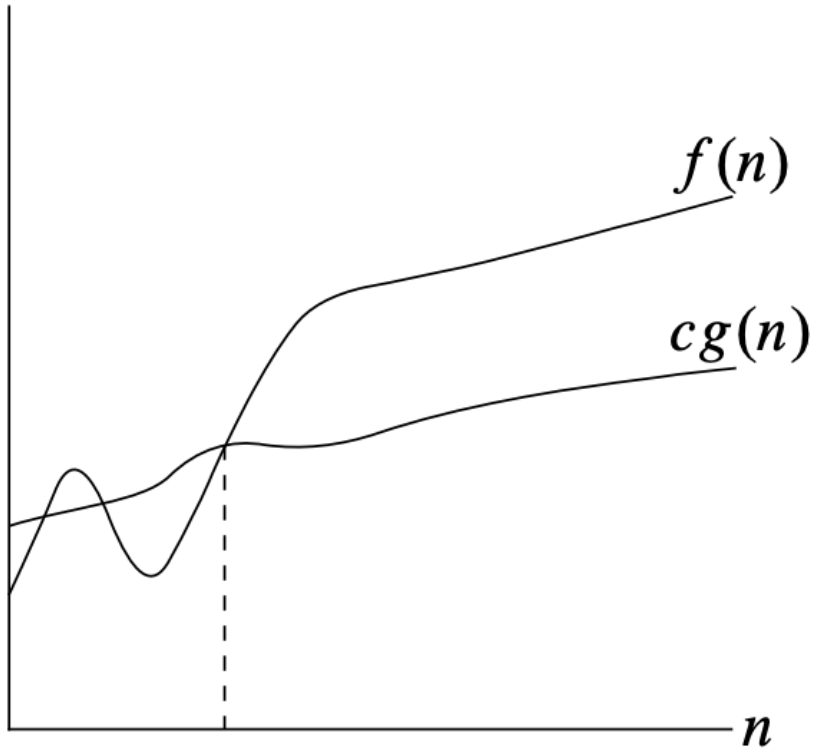
Polynomial $O(n^k)$ $k \geq 0$

$O(1) \subset O(\log_2 n) \subset O(n) \subset O(n^2) \subset O(2^n) \subset O(n!)$

BIG OMEGA & BIG THETA

Definition: Let $f, g : \mathbb{N} \to \mathbb{R}^+$ be functions

f is $\Omega(g)$ if $\exists$ positive constants c and n_0 such that $f(n) \geq c \cdot g(n) \quad \forall \quad n \geq n_0$



$f(n) = \Omega(g(n))$

n is $\Omega(\log_2 n)$

c & n_0

such that

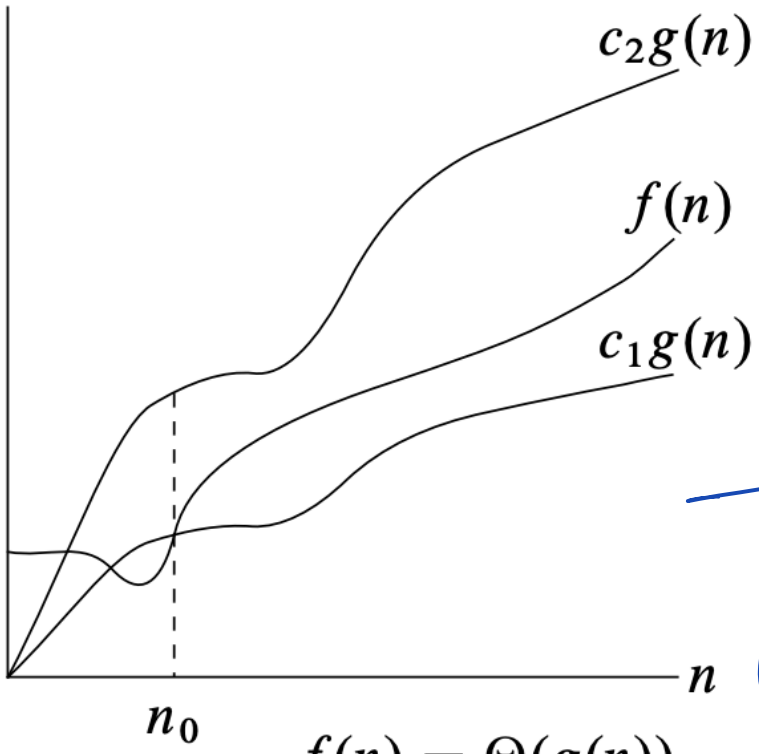
$n \geq c \log_2 n$

$c = 1, n_0 = 1$

BIG OMEGA & BIG THETA

Definition: Let $f, g : \mathbb{N} \to \mathbb{R}^+$ be functions

f is $\Theta(g)$ if $\exists$ positive constants c_1, c_2, n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$



$f(n) = \Theta(g(n))$

E.g.

$3n + 2$ is $\Theta(n)$

c_1, c_2, n_0

$c_1 \geq 2$, $c_2 = 4$, $n_0 = 2$

$$2n \leq 3n + 2 \leq 4n$$

$$\forall n \geq 2$$

EXAMPLES.

$\frac{n}{\log_2 n}$ is $O(n)$ ✓, $\Omega(n)$, $\Theta(n)$ ✗.

2^{n+2} is $O(n)$, $\Omega(n)$, $\Theta(n)$

n is $O(n^2)$, $\Omega(n^2)$, $\Theta(n^2)$

SIGACT NEWS APR-JUN 1976

BIG OMICRON AND BIG OMEGA AND BIG THETA

Donald E. Knuth
Computer Science Department
Stanford University
Stanford, California 94305

Well, I think I have beat this issue to death, knowing of no other arguments pro or con the introduction of Ω and Θ. On the basis of the issues discussed here, I propose that members of SIGACT, and editors of computer science and mathematics journals, adopt the O, Ω, and Θ notations as defined above, unless a better alternative can be found reasonably soon. Furthermore I propose that the relational notations of Hardy be adopted in those situations where a relational notation is more appropriate.